

UNIVERSIDAD SANTO TOMÁS SECCIONAL TUNJA
Facultad de Ingeniería de Sistemas

DESARROLLO SEGURO DE SOFTWARE

Módulo final · La interfaz en React y la estrategia de pruebas · API_TEATRO

ESPACIO ACADÉMICO:
Seguridad en el Desarrollo de Software

PRESENTADO A:
Sergio Arley Puerto Moreno

AUTORES:
Juan Sebastián Bonilla León
Juan Camilo Calderón Delgado
Silvana Sofía Siza Soriano
Cristian Emanuel Hernández Araque
Cristian Rodrigo Amaya Torres

2026

1. Qué pedía el módulo

El módulo final cierra el proyecto con una interfaz web desacoplada: una SPA en **React 19, Vite 8, React Router 7 y Bootstrap 5.3** en `localhost:5173` que consume la API en Express de `localhost:3000` con JWT. La guía pide:

- un panel por rol, con rutas protegidas, la sesión en `sessionStorage` y cierre de sesión cuando el token vence o se revoca;
- que el navegador no lleve ningún secreto: en las variables `VITE_` solo va la dirección de la API;
- las pruebas finales de la sección 8 del README, de la autenticación hasta el `build` y el `lint`;
- la **estrategia de pruebas**: SCA con `npm audit`, SAST con Semgrep, DAST con OWASP ZAP y pruebas de API con Swagger UI.

Cómo lo adaptamos al teatro

La guía describe tres roles: el administrador, un rol profesional que atiende una agenda y el usuario final que reserva. En el teatro quedaron así:

- El administrador tiene indicadores, cuentas, eventos y funciones, asistentes, localidades, boletas y recuperación administrativa.
- La taquilla, en el lugar del rol profesional, tiene la agenda de funciones por mes, semana y día, y el control de boletas: cobrar, vender y registrar la entrada en la puerta.
- El asistente, que es el usuario final, maneja sus boletas (comprar sobre el plano, pagar de forma simulada y cancelar), sus datos y la seguridad de su cuenta.

Repositorio: https://github.com/JuanK2550/API-TEATRO (rama `main`, commits `c8a747b` a `03e394a`). La interfaz está en `teatro-frontend/`.

Todas las figuras salen de ejecuciones reales del 13 y 14 de noviembre de 2026. El correo del administrador aparece como `<ADMIN_EMAIL>`, el JWT como `eyJhbGci…` (también en el diálogo Authorize y en el bloque Curl de Swagger), y el token de recuperación y los códigos de respaldo como `•••`. Se enmascararon en la propia página antes de capturar.

2. Una sola interfaz

Hasta el Bloque 11 el teatro tenía una página propia sin framework, la Sala, que mostraba la cartelera y permitía registrarse y comprar. Al construir el panel en React quedaban dos interfaces para la misma API, así que dejamos solo la de React. Pero no la borramos sin más: la Sala tenía cuatro cosas que el panel no tenía. Si se perdían, nadie podría registrarse

ni ver la cartelera sin cuenta. Antes de retirarla pasamos a React la cartelera pública, la ficha de cada evento, el registro de asistentes y el plano de butacas.

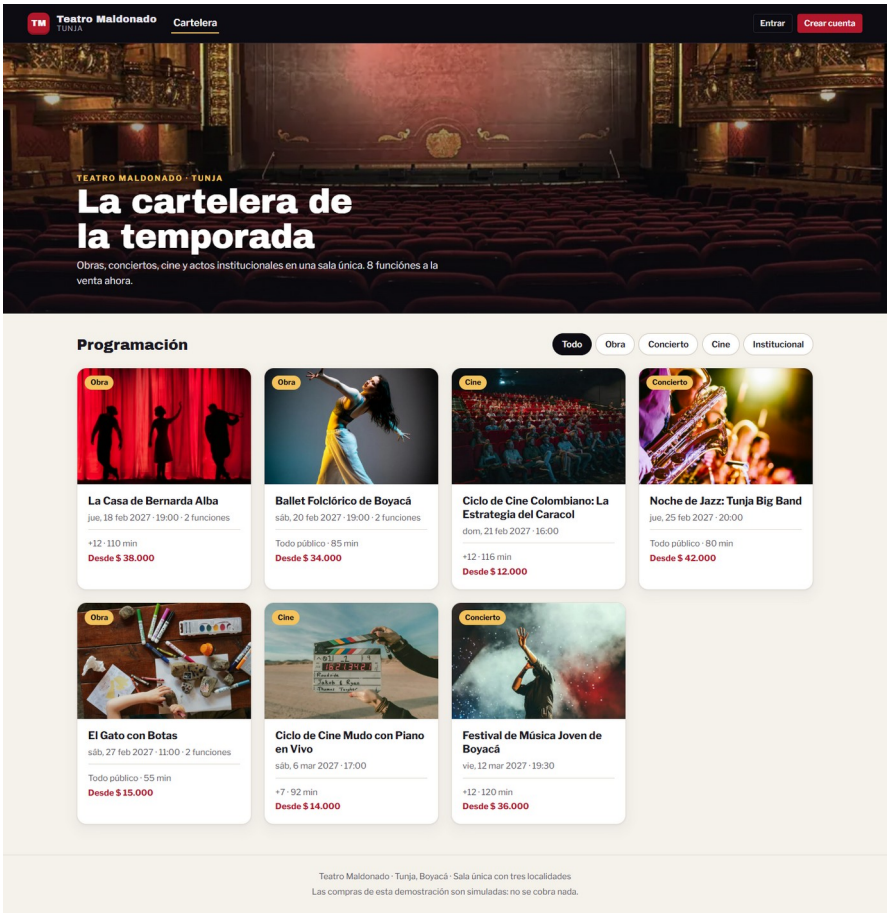


Figura 1. La cartelera pública en /, sin cuenta, con filtro por tipo. Solo muestra lo que el Bloque 11 dejó como catálogo público.

Si alguien pulsa «Comprar» sin sesión, primero entra o se registra y después vuelve a esa misma compra. En el backend retiramos el montaje de la Sala, su política de contenido y la ruta `/sala/config.js`. La prueba 34 del Bloque 11 ahora comprueba que esa ruta responde 404 y no entrega ninguna clave.

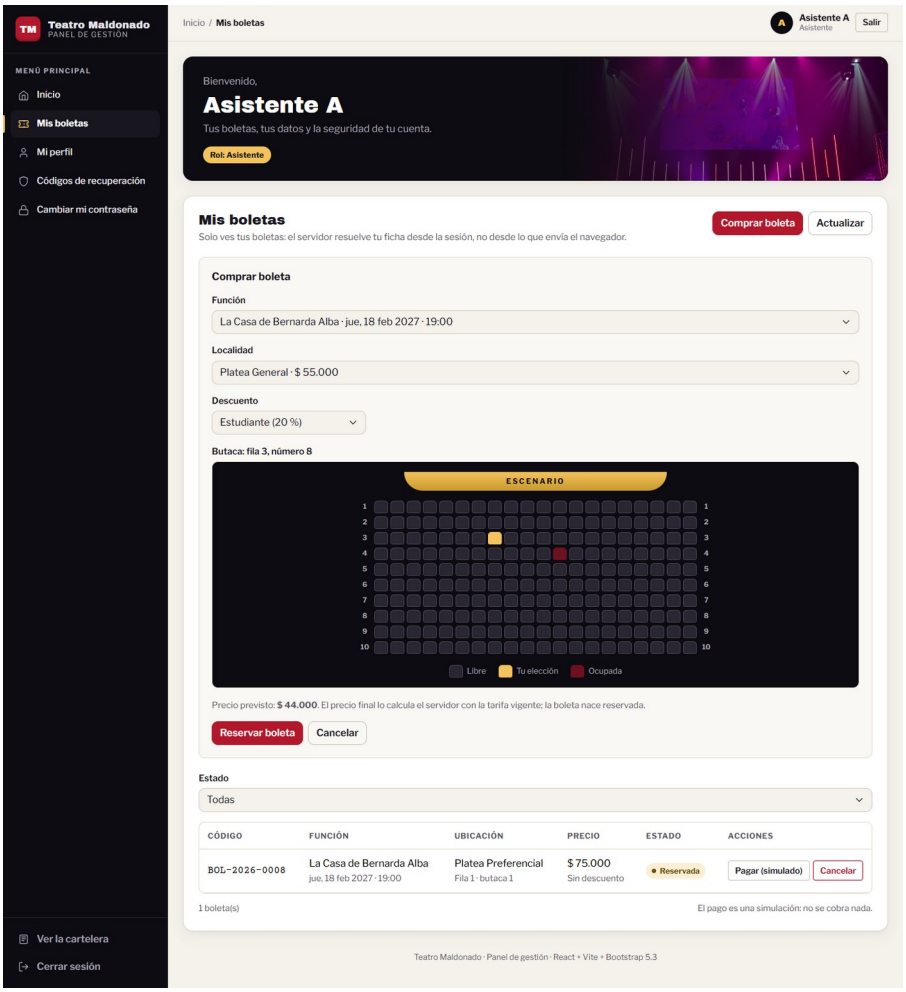


Figura 2. La compra del asistente sobre el plano de butacas. El precio que se muestra es una previsión: el definitivo lo calcula el servidor.

3. El panel por rol

Administrador

El inicio del administrador muestra siete indicadores, las boletas por estado y por mes y las próximas funciones. Para eso agregamos al backend `GET /api/estadisticas/resumen`, que devuelve solo cifras agregadas. La prueba 4 de `pruebas-panel.js` comprueba que no trae ningún correo, nombre ni documento, y la 5 que los ingresos coinciden con la suma calculada en PostgreSQL.

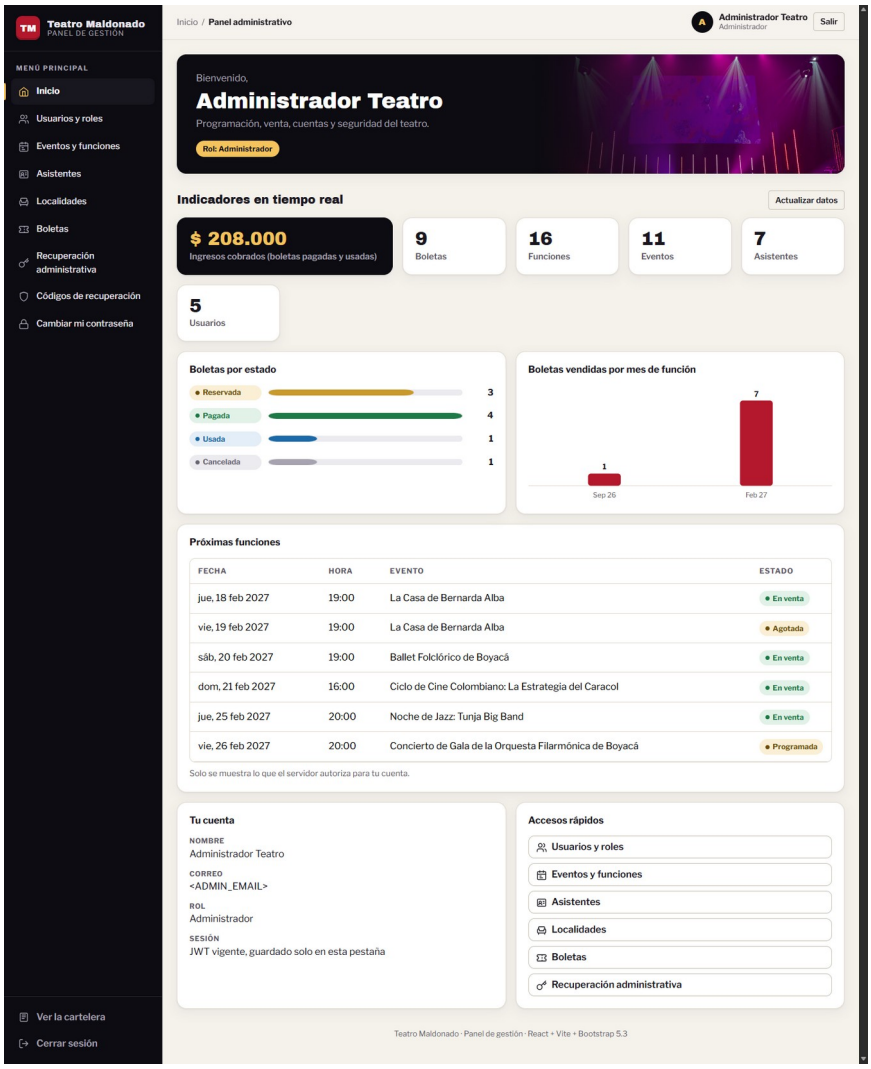


Figura 3. Inicio del administrador: indicadores sin datos personales y su cuenta con el correo enmascarado.

Para gestionar cuentas agregamos cuatro endpoints más: listar con filtros y paginación, buscar, editar nombre y correo, y activar o desactivar. Las pruebas comprueban tres cosas:

- los campos colados en el cuerpo se ignoran: si alguien manda `rol` o `activo` al editar, no cambian;
- desactivar una cuenta sube su versión de sesión, así que su token deja de servir en el acto;
- nadie puede desactivarse a sí mismo ni dejar el teatro sin un administrador activo.

La recuperación administrativa del Bloque 11 ahora tiene pantalla propia. El administrador busca la cuenta, confirma que verificó la identidad del titular y recibe el token una sola vez. La persona lo canjea sin sesión y vuelve al acceso con un aviso:

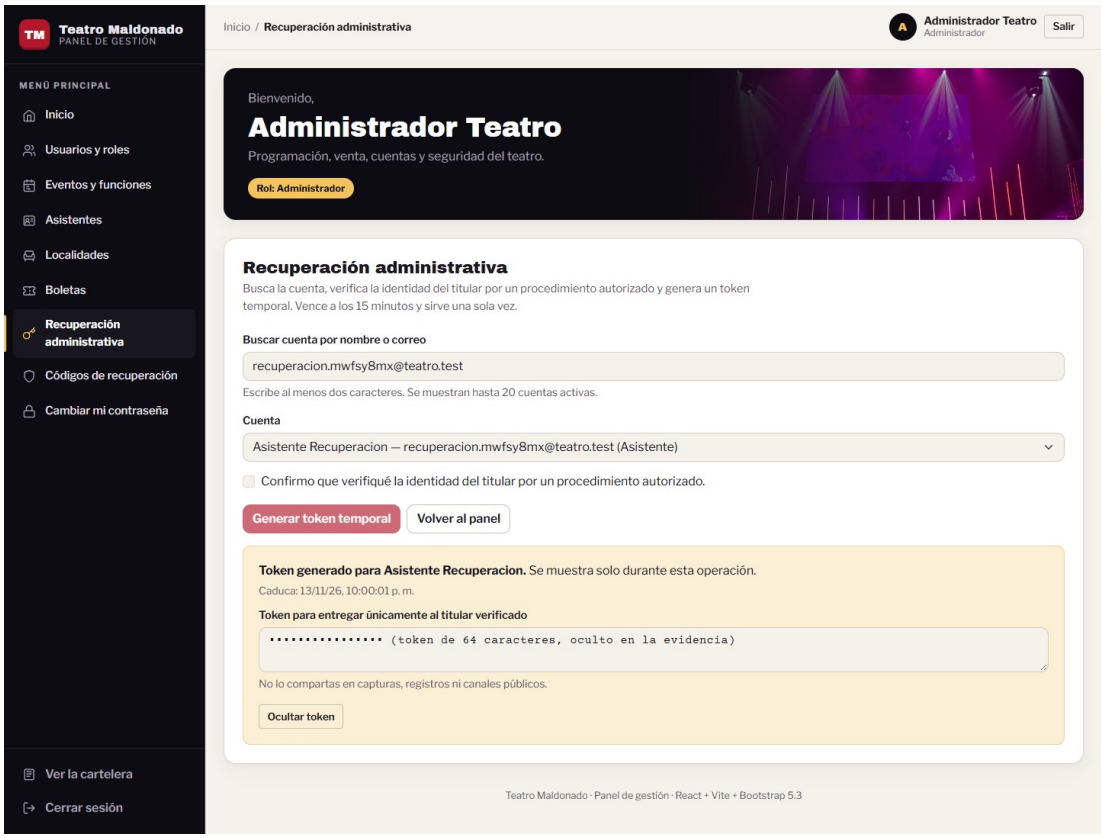


Figura 4. El token generado por el administrador, enmascarado, con su aviso de que solo se muestra una vez y vence en 15 minutos.

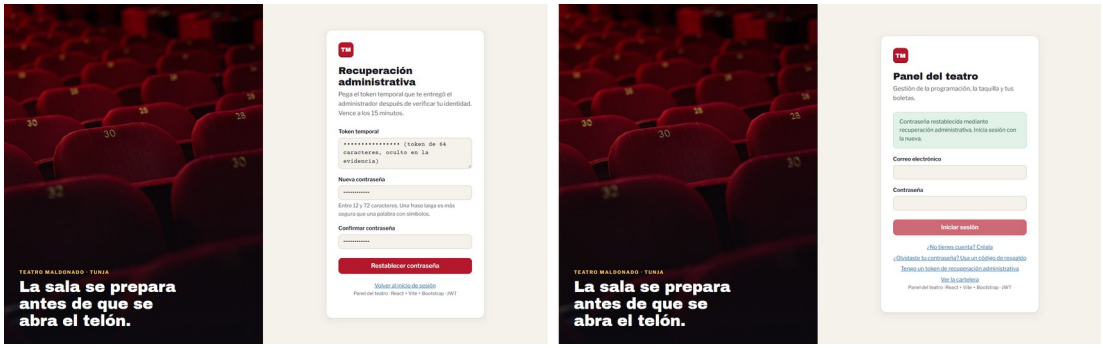


Figura 5. El canje del token sin sesión y la vuelta al acceso con el aviso de contraseña restablecida.

Esto solo fue posible porque **las rutas de restablecimiento ya no piden API Key**. Era la decisión que el Bloque 11 dejó pendiente para cuando existiera una SPA. Siguen protegidas por el propio secreto, que es de un solo uso y vence, y por el límite de diez intentos por ventana.

Taquilla

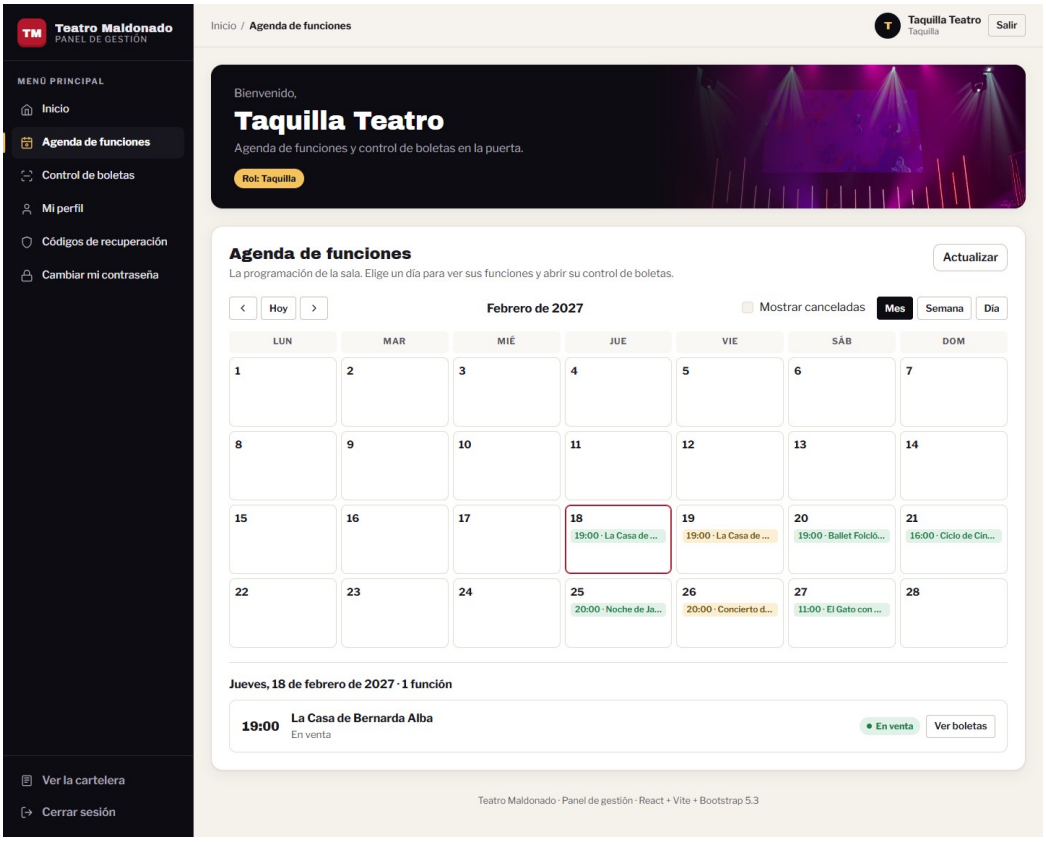


Figura 6. La agenda de la taquilla por mes; también se puede ver por semana y por día.

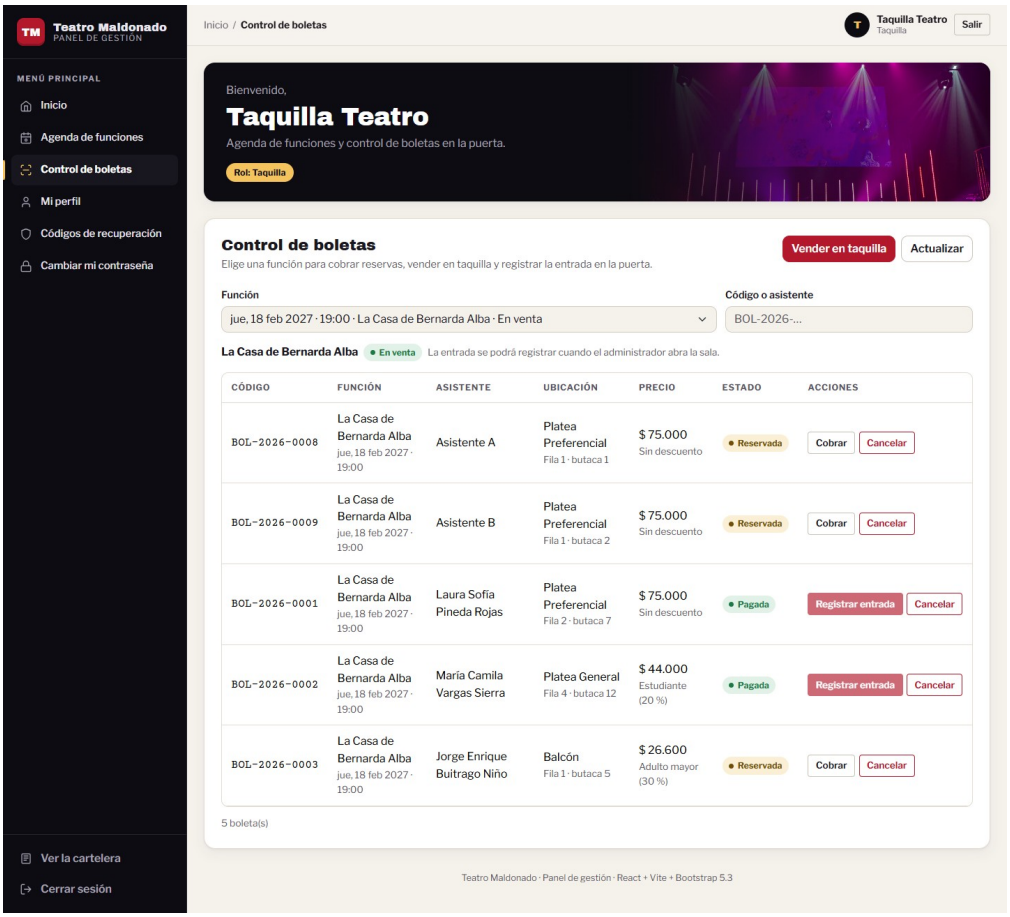


Figura 7. Control de boletas: cobrar, vender en taquilla y registrar la entrada. «Registrar entrada» queda deshabilitado mientras la función no está en curso.

4. Las pruebas finales vistas desde el panel

La sección 8 del README propone doce pruebas finales. La mayoría ya estaban cubiertas por las baterías automáticas, pero quisimos verlas también desde la interfaz, que es como las ve el usuario. Al hacerlas así aparecieron fallos de la interfaz que ninguna revisión anterior había visto (UI-04 a UI-06 en la sección 6).

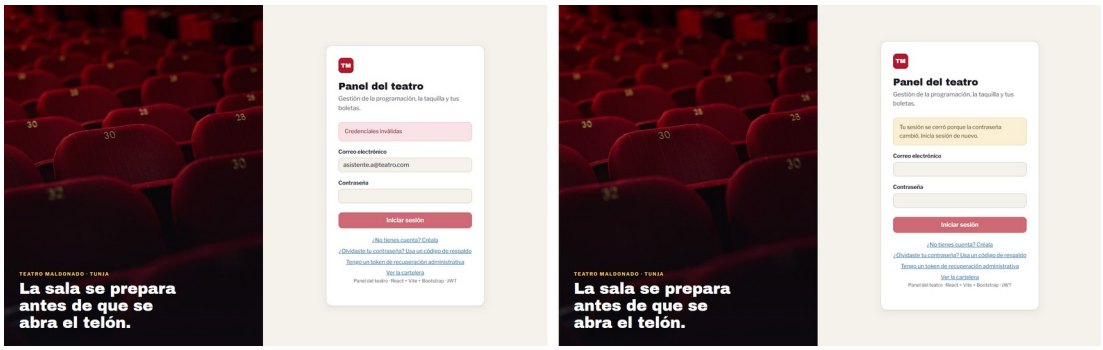


Figura 8. Izquierda: login con contraseña incorrecta, «Credenciales inválidas». Derecha: la contraseña se cambió desde otra pestaña y el panel cerró la sesión con su aviso.

El login fallido responde lo mismo con un correo inexistente que con una contraseña equivocada (pruebas 1 y 2 de `pruebas-panel.js`), así que no sirve para averiguar qué

cuentas existen. La sesión revocada es la versión de sesión del Bloque 11 vista desde el navegador: el token viejo recibe 401, el panel lo detecta, borra la sesión de `sessionStorage` y lleva al acceso.

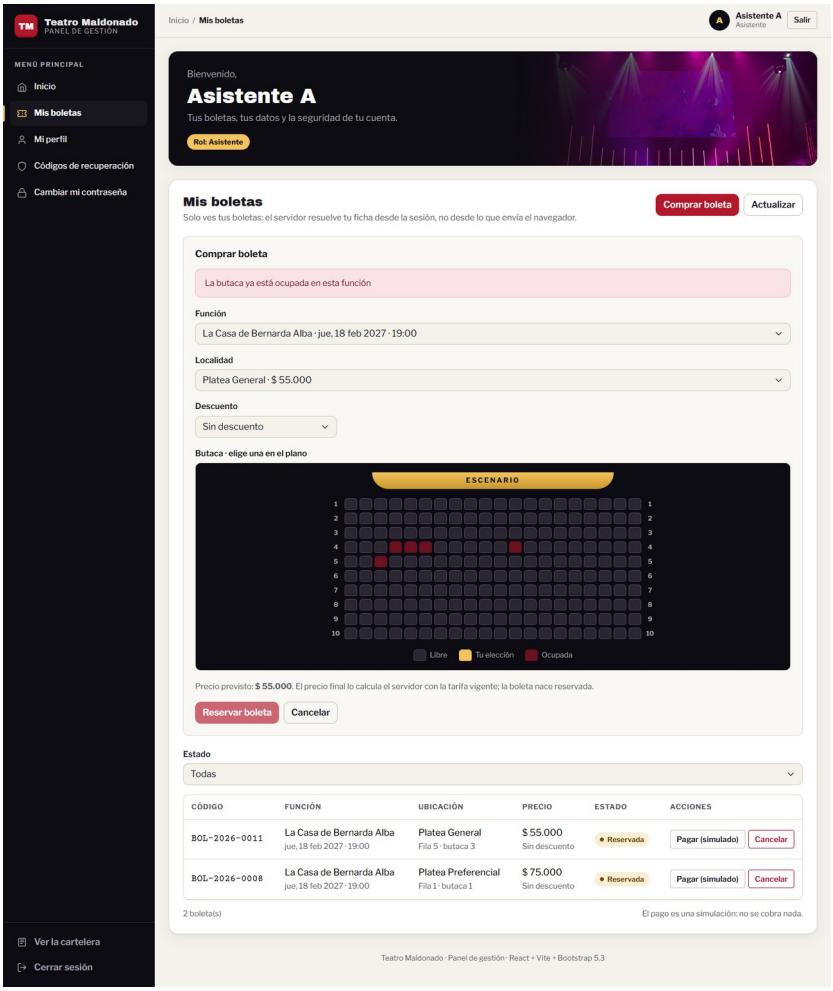


Figura 9. Otra persona compró la butaca elegida antes de confirmar: la API responde 409 y el panel suelta la butaca en el plano.

La butaca ocupada es nuestro equivalente a la prueba de la guía que reserva en un horario ya ocupado. El conflicto lo detienen dos capas: la lógica del servicio y el índice único de la base, que impide dos boletas activas para la misma butaca en la misma función aunque dos peticiones lleguen a la vez.



Figura 10. El registro del navegador en las pruebas 43 a 47: la ruta donde quedó y el mensaje que mostró la interfaz en cada caso.

La tarjeta también registra la prueba de ruta protegida: un asistente que abre `/panel/usuarios` termina en su inicio. `RutaProtegida` solo mejora la experiencia; la protección real es la del backend, que responde 403 aunque alguien fuerce la petición (figura 12).

5. La estrategia de pruebas

Pruebas de API con Swagger UI

La cuarta técnica de la estrategia pide verificar autenticación, autorización y validaciones desde Swagger UI. En los bloques anteriores evitábamos pulsar Execute para que no quedara un token a la vista; aquí lo hicimos tapando el token en la página antes de capturar.

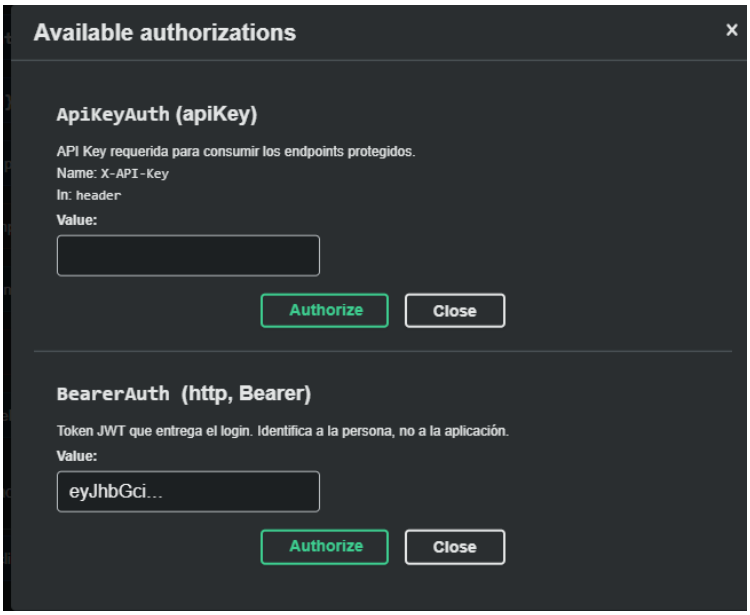


Figura 11. El diálogo Authorize con BearerAuth y el token enmascarado. La API Key solo la piden las integraciones privadas.

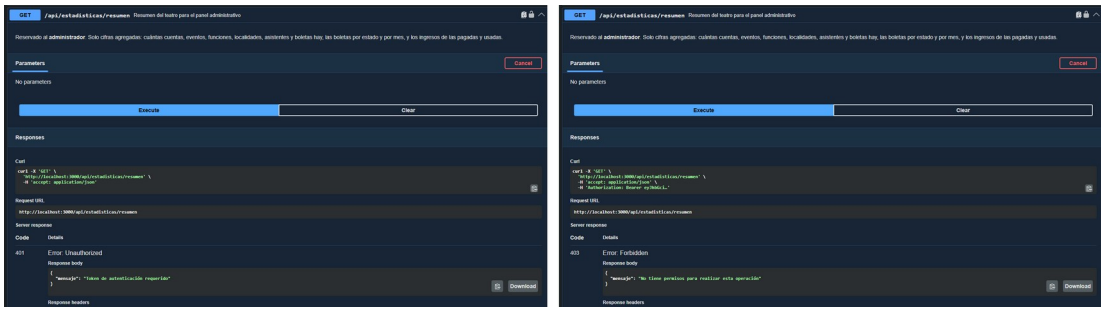


Figura 12. La misma operación sin token (401) y con el token de un asistente (403). El Curl de la derecha lleva el token enmascarado.

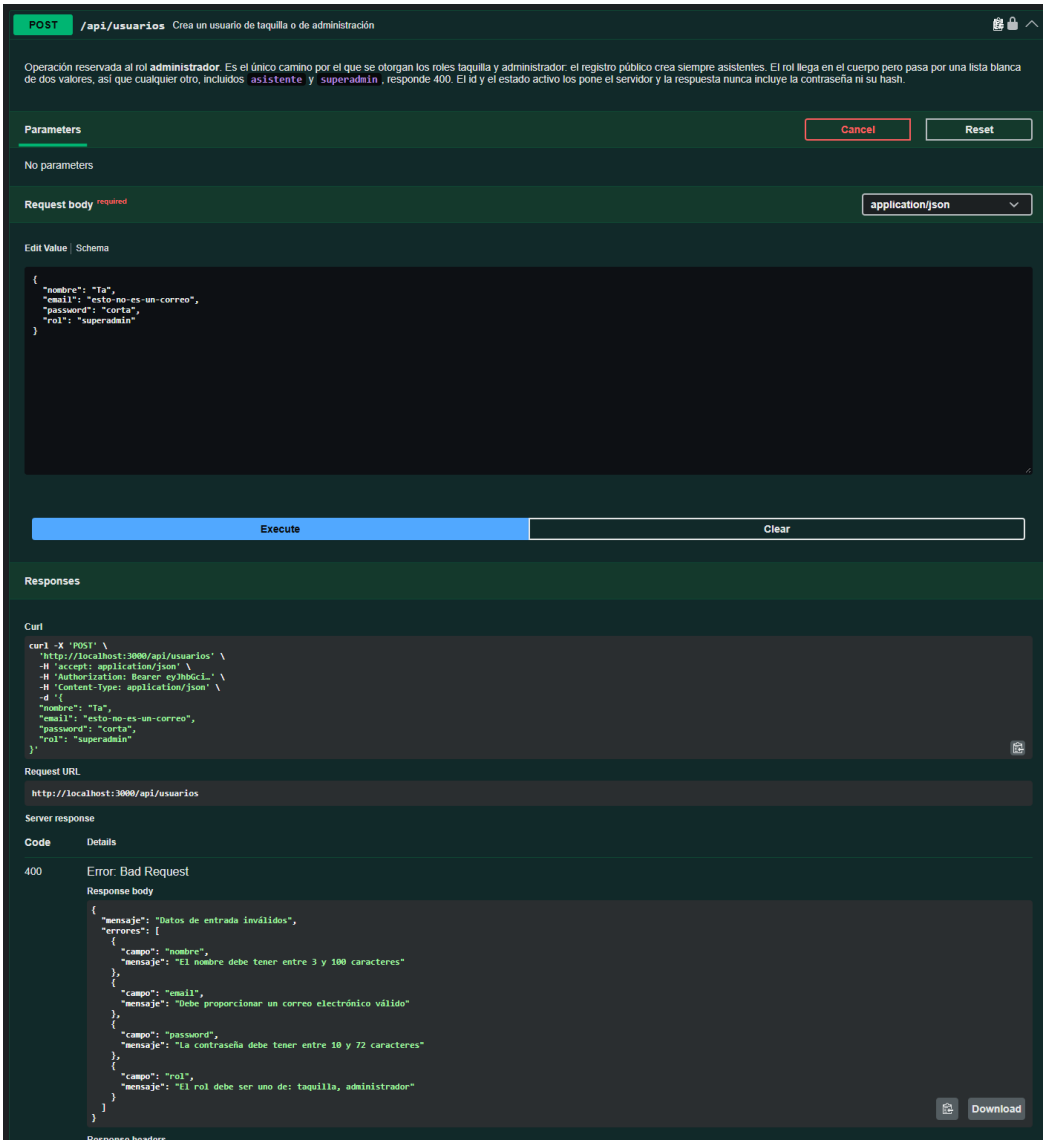


Figura 13. POST /api/usuarios con un cuerpo inválido: 400 con el error de cada campo, incluido un rol que no está en la lista blanca.

La figura 13 muestra dos cosas: la validación responde campo por campo, y el rol solo puede ser `taquilla` o `administrador` (el registro público crea siempre asistentes). Intentar crear un `superadmin` no llega a la base.

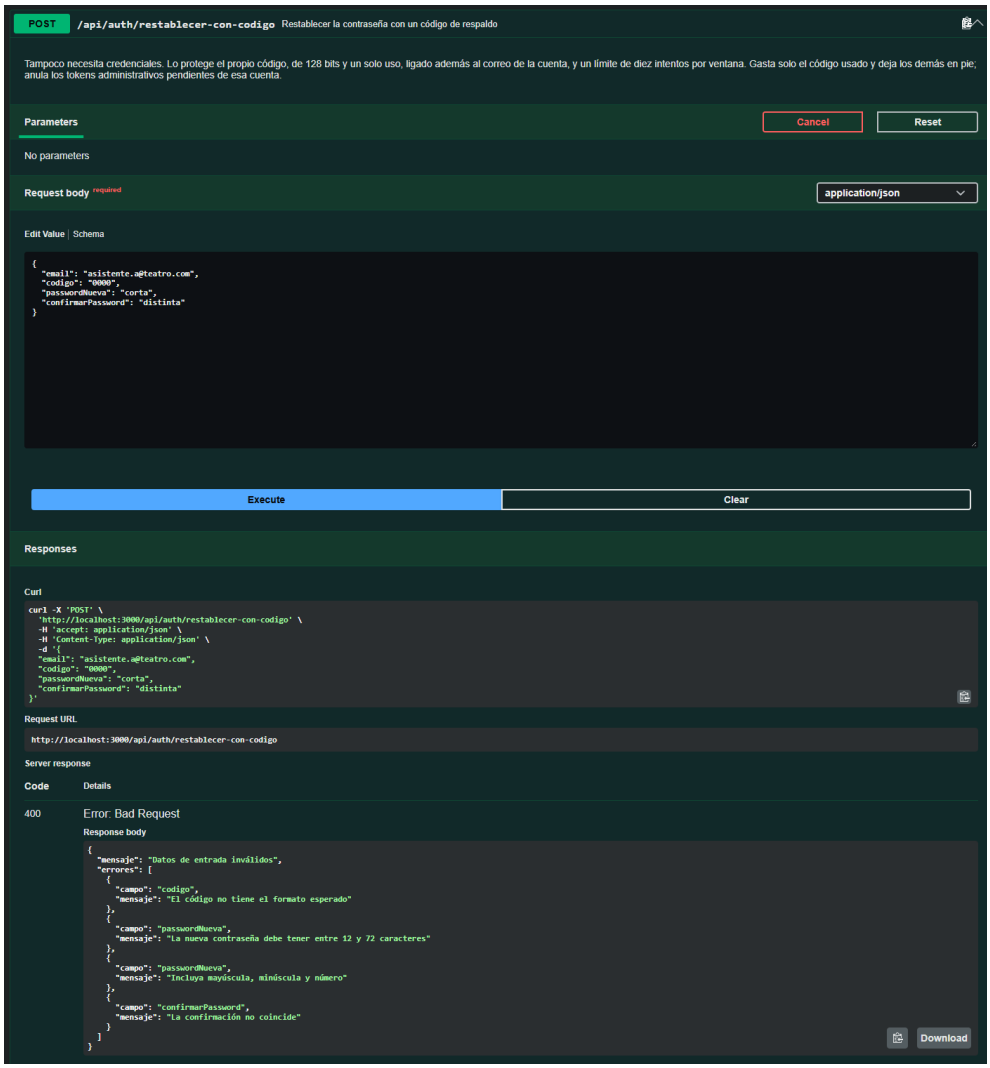


Figura 14. Restablecer con un código sin API Key: la petición llega al validador y responde 400 por el cuerpo, no 401 por la clave.

La operación correcta, el resumen como administrador con 200, también está en las evidencias (captura 39). Con esto quedan los cinco casos: sin token, rol equivocado, cuerpo inválido, éxito y restablecimiento sin clave.

SCA y SAST

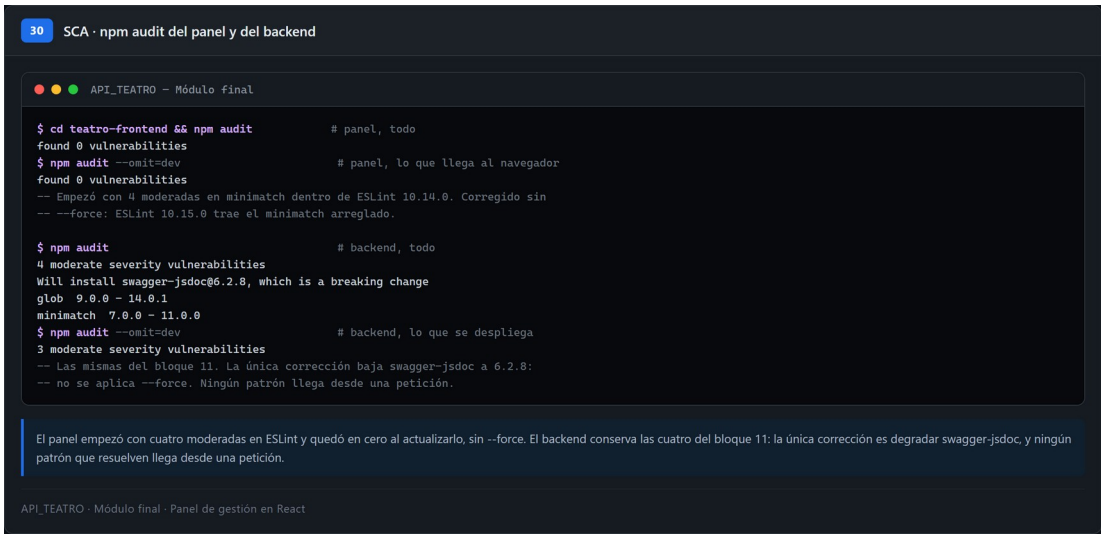


Figura 15. SCA con npm audit: la interfaz en 0; el backend con las 4 moderadas que ya conocíamos del Bloque 11.

Las cuatro moderadas del backend vienen de `glob` y `minimatch` a través de `swagger-jsdoc` y `nodemon`. La única corrección que ofrece npm es bajar `swagger-jsdoc` a una versión anterior. Además, ninguna es alcanzable desde una petición: solo resuelven el patrón fijo de las rutas al arrancar.



Figura 16. SAST con Semgrep, con las reglas de React además de las de JavaScript: 0 hallazgos en 133 archivos.

DAST con OWASP ZAP, autenticado

Un DAST sin sesión solo ve la API desde fuera: la mayoría de las rutas responde 401 y el escáner no llega a las operaciones que importan. Eso pasó en nuestro primer intento, y lo detectamos con una **petición de control**: antes de escanear, `/api/estadisticas/resumen`, que sin token responde 401, tiene que dar 200. Si no da 200, el escaneo no está autenticado.

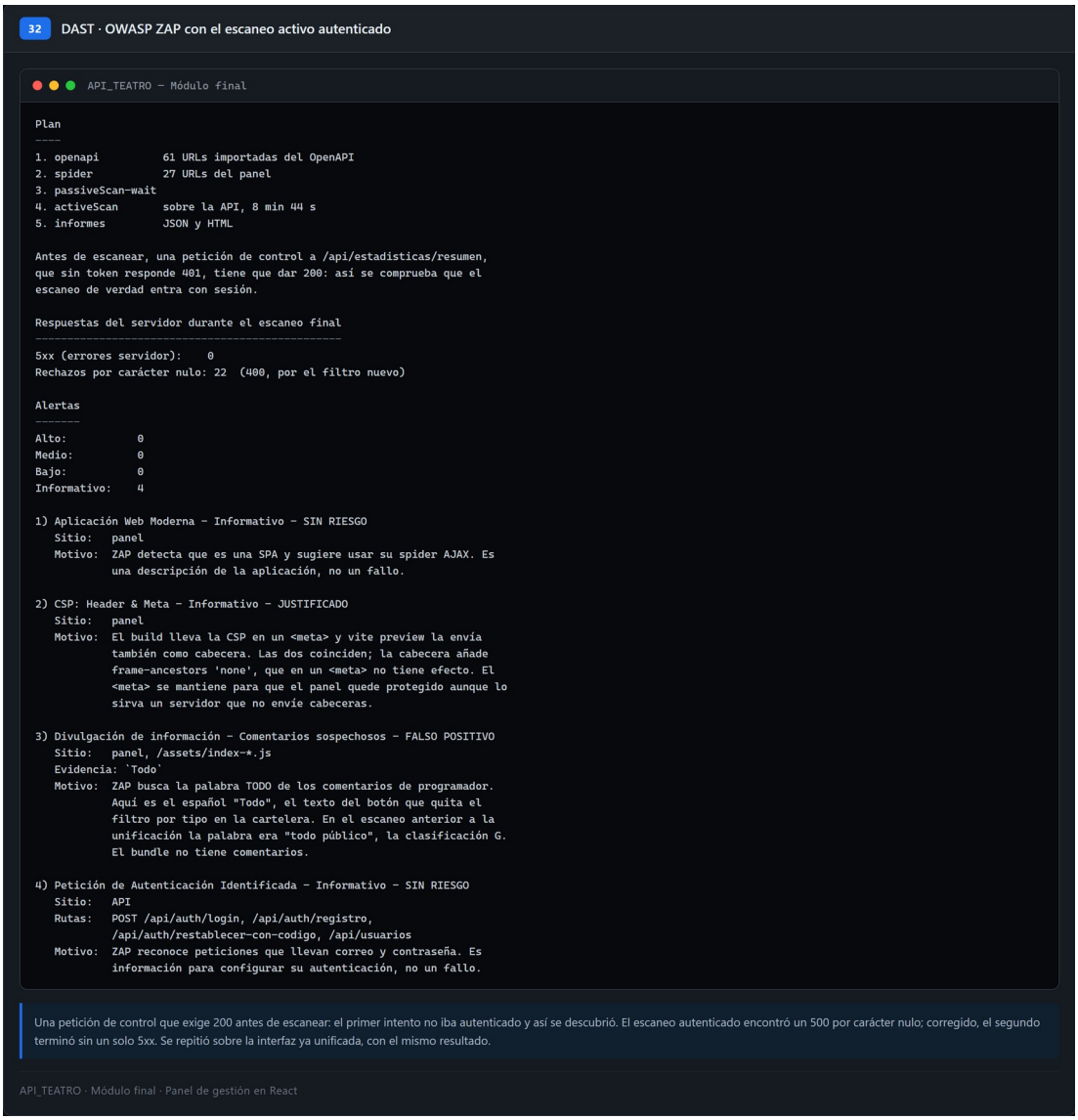
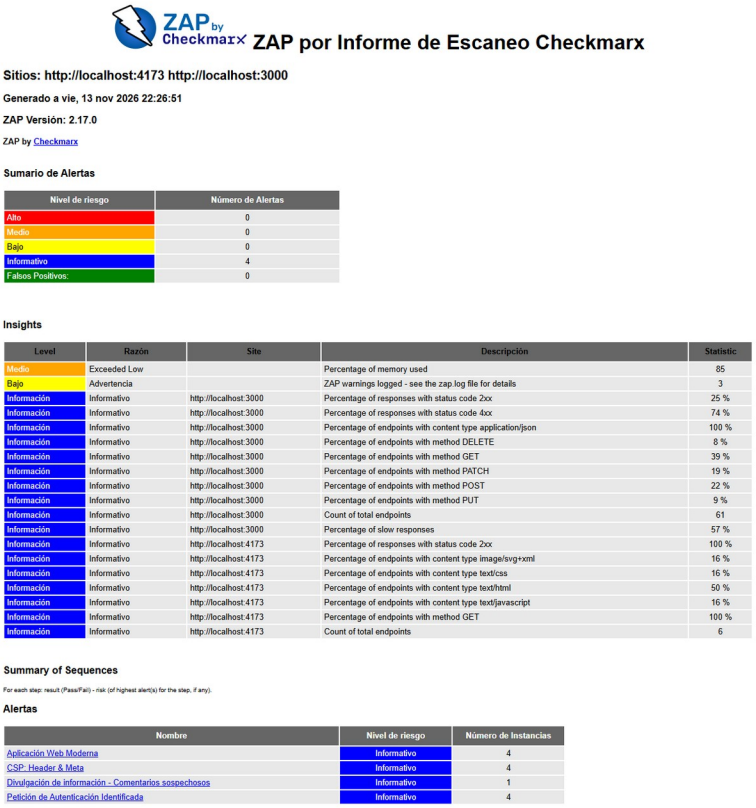


Figura 17. El plan del escaneo, la petición de control, cero respuestas 5xx y las cuatro alertas informativas con su valoración.



ZAP por Informe de Escaneo Checkmarx

Sitios: http://localhost:4173 http://localhost:3000

Generado a vie, 13 nov 2026 22:26:51

ZAP Versión: 2.17.0

ZAP by Checkmarx

Sumario de Alertas

Nivel de riesgo	Número de Alertas
Alto	0
Medio	0
Bajo	0
Informativo	4
Falsos Positivos:	0

Insights

Level	Razón	Site	Descripción	Statistic
Medio	Exceeded Low		Percentage of memory used	85
Bajo	Advertencia		ZAP warnings logged - see the zap.log file for details	3
Información	Informativo	http://localhost:3000	Percentage of responses with status code 2xx	25 %
Información	Informativo	http://localhost:3000	Percentage of responses with status code 4xx	74 %
Información	Informativo	http://localhost:3000	Percentage of endpoints with content type application/json	100 %
Información	Informativo	http://localhost:3000	Percentage of endpoints with method DELETE	8 %
Información	Informativo	http://localhost:3000	Percentage of endpoints with method GET	39 %
Información	Informativo	http://localhost:3000	Percentage of endpoints with method PATCH	19 %
Información	Informativo	http://localhost:3000	Percentage of endpoints with method POST	22 %
Información	Informativo	http://localhost:3000	Percentage of endpoints with method PUT	9 %
Información	Informativo	http://localhost:3000	Count of total endpoints	61
Información	Informativo	http://localhost:3000	Percentage of slow responses	57 %
Información	Informativo	http://localhost:4173	Percentage of responses with status code 2xx	100 %
Información	Informativo	http://localhost:4173	Percentage of endpoints with content type image/svg+xml	16 %
Información	Informativo	http://localhost:4173	Percentage of endpoints with content type text/css	16 %
Información	Informativo	http://localhost:4173	Percentage of endpoints with content type text/html	50 %
Información	Informativo	http://localhost:4173	Percentage of endpoints with content type text/javascript	16 %
Información	Informativo	http://localhost:4173	Percentage of endpoints with method GET	100 %
Información	Informativo	http://localhost:4173	Count of total endpoints	6

Summary of Sequences

For each step: result (Pass/Fail) - risk (of highest alert(s)) for the step, if any).

Alertas

Nombre	Nivel de riesgo	Número de Instancias
Aplicación Web Moderna	Informativo	4
CSP: Header & Meta	Informativo	4
Divulgación de información - Comentarios sospechosos	Informativo	1
Petición de Autenticación Identificada	Informativo	4

Figura 18. El informe HTML real de ZAP: 0 altas, 0 medias, 0 bajas y 4 informativas.

Las cuatro informativas no son fallos: ZAP detecta que es una SPA, que la CSP va en cabecera y en `<meta>` a la vez (a propósito, porque `frame-ancestors` solo funciona en cabecera), que hay peticiones con correo y contraseña, y una palabra «Todo» que confunde con un comentario de programador (es el botón que quita el filtro de la cartelera). El escaneo final se repitió después de juntar las interfaces, sobre lo que se entrega.

El hallazgo del DAST: un 500 por un carácter nulo

El escaneo autenticado sí encontró algo real. ZAP envió un `\u0000` en la búsqueda de cuentas y la API respondió 500: PostgreSQL no admite el carácter nulo en un texto, y ese error terminaba como error del servidor. Lo mismo pasaba en el registro, que es anterior a este módulo.

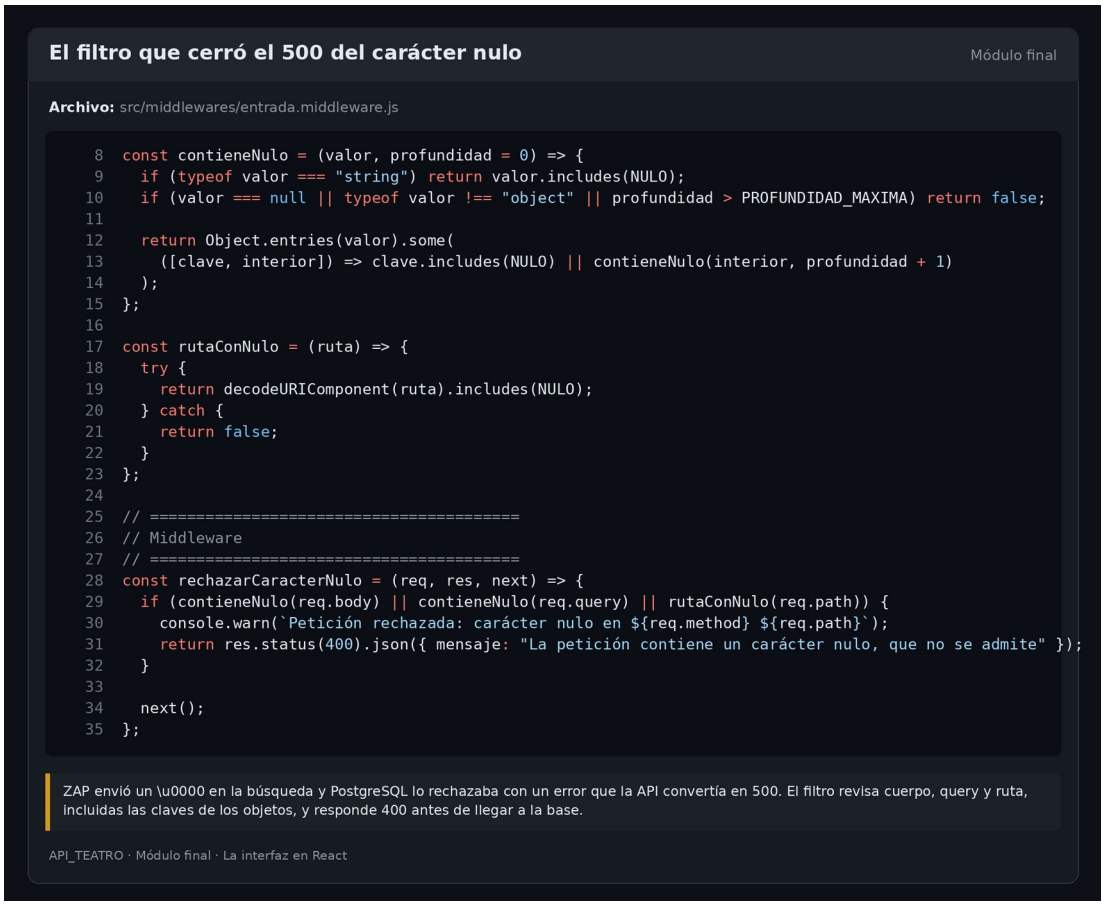


Figura 19. El filtro de entrada que rechaza el carácter nulo con 400 antes de llegar a la base.

En el escaneo final hubo 22 peticiones con carácter nulo y todas recibieron 400. No hubo ninguna respuesta 5xx en todo el escaneo.

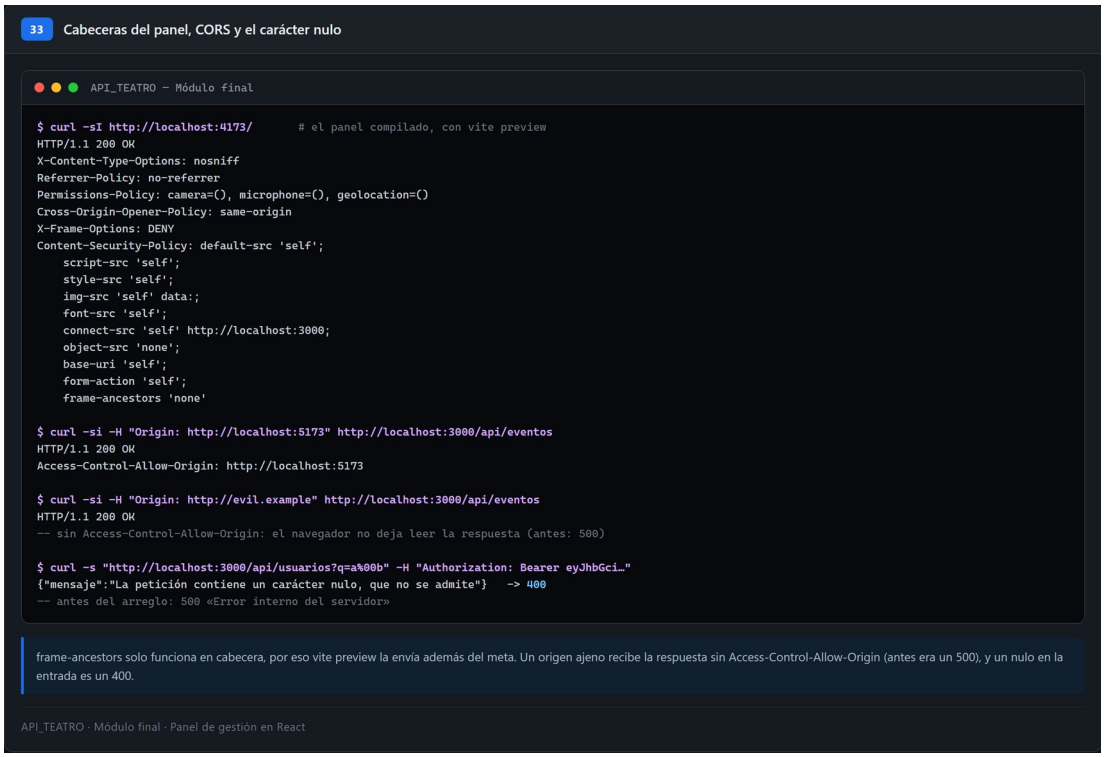


Figura 20. Las cabeceras del panel compilado con su CSP estricta, CORS con el origen del panel y uno ajeno, y el carácter nulo rechazado.

6. Lo que encontramos y corregimos

Estos son los hallazgos del módulo, con lo que hicimos en cada uno.

ID	Hallazgo	Qué hicimos
SEC-02	Un origen no permitido por CORS provocaba un 500	Sale sin la cabecera CORS y el navegador bloquea
DAST-03	El primer escaneo activo no iba autenticado	Petición de control que exige 200 y escaneo repetido
DAST-04	Un carácter nulo llegaba a PostgreSQL y daba 500	Filtro de entrada que responde 400
UI-03	Tras cerrar sesión, la siguiente persona entraba en la última página de la anterior	Un cierre voluntario ya no guarda página de vuelta
UI-04	Comprar y vender desde la interfaz respondía 400: faltaba asistenteId en el pedido	Corregido; compra y venta verificadas en el navegador
UI-05	Los errores de validación salían sin el campo que fallaba	El panel lee el formato de errores del proyecto
UI-06	Tras un 409, la butaca seguía elegida en el plano	El conflicto la suelta
DOC-01	El README decía que un código de respaldo se podía adivinar	Corregido: son 128 bits
DOC-02	Swagger hablaba de API Key en 35 respuestas 401 que solo usan JWT	Corregido

El más serio es UI-04: **comprar desde la interfaz nunca había funcionado**. Nuestras primeras capturas mostraban la pantalla de compra con su precio previsto, pero ninguna revisión llegó a pulsar «Reservar». La API rechazaba bien el pedido incompleto; el error estaba en la interfaz.

7. Build, lint y la tanda completa

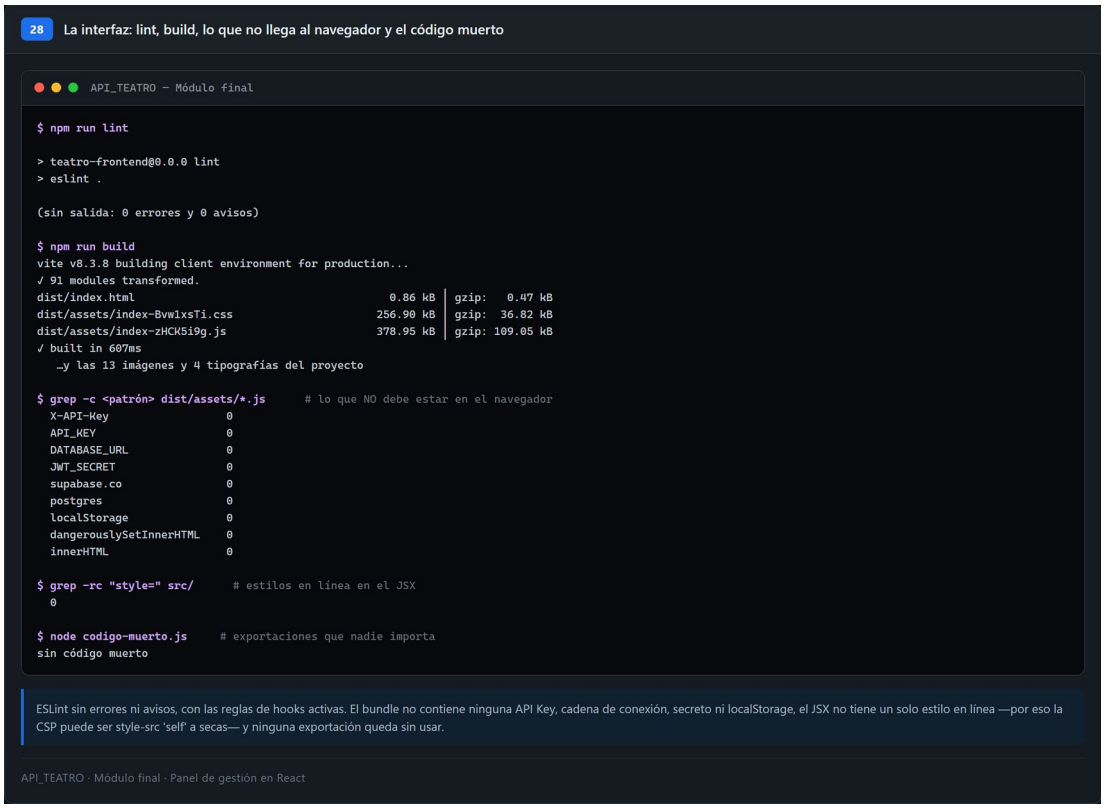


Figura 21. Lint sin errores ni avisos, el build y lo que no debe llegar al navegador: ninguna API Key, cadena de conexión, secreto ni localStorage.

El barrido del bundle es la comprobación de que el navegador no lleva secretos: `X-API-Key`, `DATABASE_URL`, `JWT_SECRET`, `supabase.co` y `localStorage` aparecen cero veces. El JSX no tiene estilos en línea, y por eso la CSP puede quedarse en `style-src 'self'`. El `.env.example` de la interfaz solo tiene `VITE_API_URL`, con la advertencia de que ahí nunca va un secreto.



Figura 22. pruebas-panel.js: las 43 pruebas de los endpoints nuevos, la recuperación sin API Key, el carácter nulo, CORS y el contrato en Swagger.

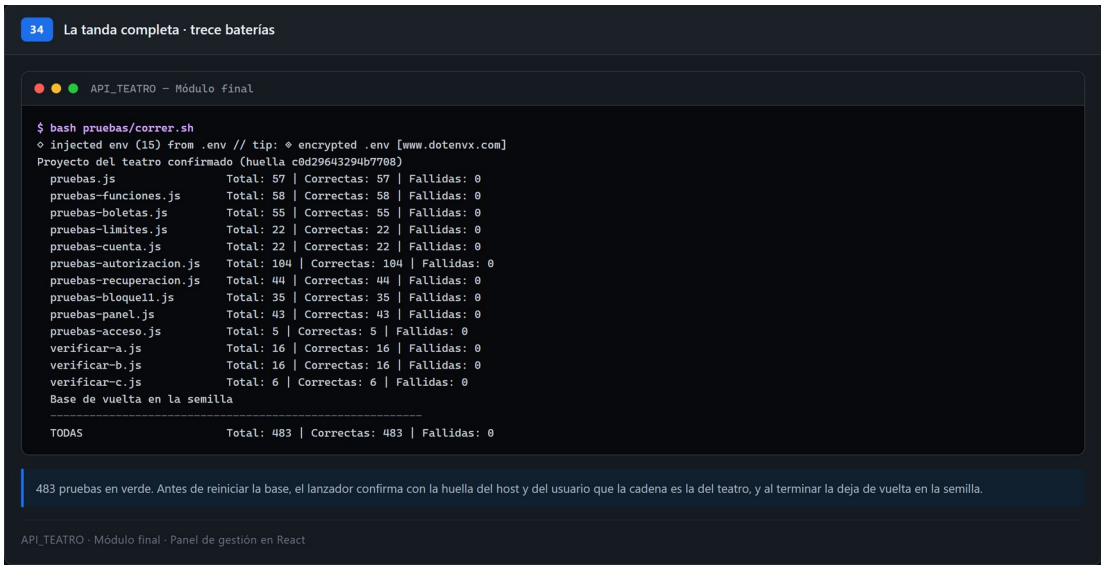


Figura 23. La tanda completa: 483 de 483 en trece baterías, con la salvaguarda por huella antes de reiniciar y la base de vuelta en la semilla.

Al terminar, `correr.sh` deja la base en la semilla y guarda la salida completa de cada batería. Lo agregamos porque en una corrida aparecieron tres fallos en `pruebas-panel.js` que después no pudimos reproducir: en diez corridas seguidas dio 43 de 43. Si vuelve a pasar, quedará el registro para saber qué falló.

8. La entrega en el repositorio

Todo el proyecto queda en https://github.com/JuanK2550/API-TEATRO, rama `main`, con un README para cada parte: `database/README.md` para la base de datos, el `README.md` principal para el backend y `teatro-frontend/README.md` para la interfaz.

Los cambios quedaron en seis commits en `main`, separados por motivo:

- `c8a747b`: la interfaz en React reemplaza a la Sala;
- `5c1c515`: el resumen del panel, la gestión de cuentas, el restablecimiento sin clave y el filtro del carácter nulo;
- `de6c4ed`: la batería del panel y el Bloque 11 ajustado al contrato nuevo;
- `15bb17f`: el informe del módulo, los análisis de seguridad y el README;
- `ea5f4d9`: los scripts de shell siempre con finales LF, para que `correr.sh` funcione al clonar en Windows;
- `03e394a`: el DAST repetido sobre la interfaz unificada y la tanda que deja la base en la semilla.

Ninguno incluye un `.env` (solo los dos `.env.example`), `node_modules`, `dist` ni los informes de ZAP. Revisamos el diff completo: no hay JWT, cadenas de conexión ni bytes nulos.

9. Conclusión

Con este módulo el teatro quedó con la arquitectura que pide la guía: una SPA que solo conoce la dirección de la API, una API que decide los permisos con el JWT y una base a la que la API entra con un rol sin `BYPASSRLS`. Lo que más trabajo nos dio fue probarlo.

Cada técnica aportó algo distinto. Semgrep no encontró problemas en el código. ZAP autenticado encontró un 500 que afectaba también al registro, anterior al módulo. Las 483 pruebas automáticas pasaban, pero ninguna compraba desde la interfaz, y la compra nunca había funcionado. Swagger dejó ver que la documentación seguía pidiendo una API Key que esas rutas ya no exigían.

> **Lo que aprendimos:** una prueba solo demuestra lo que de verdad ejecuta.
> Por eso el DAST lleva una petición de control que confirma la sesión, la compra se probó pulsando «Reservar» y la ruta protegida se comprobó también contra el backend, que responde 403.